

Assignment 7: The Great Firewall of Santa Cruz - Bloom Filters, Linked Lists, Binary Trees and Hash Tables DESIGN.pdf

Description of Program:

This is a word filtering program for the GPRSC. It filters out and reports bad words parsed from stdin.

Files to be included in directory "asgn7" + pseudocode/structure:

- messages.h: Defines the mixspeak, badspeak, and goodspeak messages that are used in banhammer.c. Do not modify this.
- salts.h: Defines the primary, secondary, and tertiary salts to be used in your Bloom filter implementation. Also defines the salt used by the hash table in your hash table implementation.
- speck.h: Defines the interface for the hash function using the SPECK cipher. Do not modify this.
- speck.c: Contains the implementation of the hash function using the SPECK cipher. Do not modify this.
- ht.h: Defines the interface for the hash table ADT. Do not modify this.
- ht.c: Contains the implementation of the hash table ADT.
 - ht_lookup()
 - Searches for an entry, a node, in the hash table that contains oldspeak. A node stores oldspeak and its newspeak translation. The index of the binary search tree to perform a

look-up on is calculated by hashing the oldspeak. If the node is found, the pointer to the node is returned. Else, a NULL pointer is returned.

- `ht_insert()`
 - Inserts the specified oldspeak and its corresponding newspeak translation into the hash table. The index of the binary search tree to insert into is calculated by hashing the oldspeak.
- `ht_count()`
 - Returns the number of non-NULL binary search trees in the hash table.
- `ht_avg_bst_size()`
 - Returns the average binary search tree size. This is computed as the sum of the sizes over all the binary search trees divided by the number of non-NULL binary search trees in the hash table.
- `ht_avg_bst_height()`
 - Returns the average binary search tree size. This is computed as the sum of the heights over all the binary search trees divided by the number of non-NULL binary search trees in the hash table.
- `ht_size()`
 - Returns the hash table's size.

- `ht_print()`
 - A debug function to print out the contents of a hash table.
- `ht_delete()`
 - The destructor for a hash table. Each of the binary search trees, the underlying array of binary search tree root nodes, is freed. The pointer that was passed in should be set to NULL.
- `ht_create()`
 - The constructor for a hash table. The size parameter denotes the number of indices, or binary search trees, that the hash table can index up to. The salt for the hash table is provided in `salts.h`.
- `bst.h`: Defines the interface for the binary search tree ADT. Do not modify this.
- `bst.c`: Contains the implementation of the binary search tree ADT.
 - `bst_print(Node *root)`
 - Prints out each node in the binary search tree through an inorder traversal. This will require the use of `node_print()`.
 - `bst_insert(Node *root, char *oldspeak, char *newspeak)`
 - Inserts a new node containing the specified `oldspeak` and `newspeak` into the binary search tree rooted at `root`. Duplicates should not be inserted.
 - `bst_find(Node *root, char *oldspeak)`

- Searches for a node containing oldspeak in the binary search tree rooted at root. If a node is found, the pointer to the node is returned. Else, a NULL pointer is returned.
 - bst_size(Node *root)
 - Returns the size of the binary search tree rooted at root. The size of a tree is equivalent to the number of nodes in the tree.
 - bst_height(Node *root)
 - Returns the height of the binary search tree rooted at root.
 - bst_create(void)
 - Constructor for a binary search tree that constructs an empty tree. That means that the tree is NULL.
 - bst_delete(Node **root)
 - Destructor for a binary search tree rooted at root. This should walk the tree using a postorder traversal and delete each node of the tree.
- node.h: Defines the interface for the node ADT. Do not modify this.
- node.c: Contains the implementation of the node ADT.
 - node_print(Node *n)
 - While helpful as debug function, use this function to produce correct program output. Print out the contents of a node in the following manner:

If the node n contains oldspeak and newspeak, print out the node with this print statement:

- `printf("%s -> %s\n", n->oldspeak, n->newspeak);`

- If the node n contains only oldspeak, meaning that newspeak is null, then print out the node with

- `printf("%s\n", n->oldspeak);`

- `node_delete(Node **n)`

- The destructor for a node. Only the node n is freed. The previous and next nodes that n points to are not deleted. Since you have allocated memory for oldspeak and newspeak, remember to free the memory allocated to both of those as well. The pointer to the node should be set to NULL.

- `node_create()`

- The constructor for a node. You will want to make a copy of the oldspeak and its newspeak translation that are passed in. What this means is allocating memory and copying over the characters for both oldspeak and newspeak.

- `bf.h`: Defines the interface for the Bloom filter ADT. Do not modify this.
- `bf.c`: Contains the implementation of the Bloom filter ADT.
 - Contains `primary[]`, `secondary[]`, `tertiary[]` (for salts) and bit vector filter
 - `bf_print()`

- A debug function to print out the bits of a Bloom filter.
- bf_count()
 - Returns the number of set bits in the Bloom filter.
- bf_probe()
 - Probes the Bloom filter for oldspeak. Like with bf_insert(), oldspeak is hashed with each of the three salts for three indices. If all the bits at those indices are set, return true to signify that oldspeak was most likely added to the Bloom filter. Else, return false.
- bf_insert()
 - Takes oldspeak and inserts it into the Bloom filter. This entails hashing oldspeak with each of the three salts for three indices, and setting the bits at those indices in the underlying bit vector.
- bf_size()
 - Returns the size of the Bloom filter. In other words, the number of bits that the Bloom filter can access. This is the length of the underlying bit vector.
- bf_delete()
 - Takes in double pointer bf
 - The destructor for a Bloom filter. Should free any memory allocated by the constructor and null out the pointer that was passed in.

- `bf_create(size)`
 - The constructor for a Bloom filter. The primary, secondary, and tertiary salts that should be used are provided in `salts.h`.
- `bv.h`: Defines the interface for the bit vector ADT. Do not modify this.
- `bv.c`: Contains the implementation of the bit vector ADT.
 - `bv_get_bit()`
 - Returns the i th bit in the bit vector. If i is out of range, return false. Otherwise, return false if the value of bit i is 0 and return true if the value of bit i is 1.
 - `bv_delete()`
 - The destructor for a bit vector. Remember to set the pointer to NULL after the memory associated with the bit vector is freed.
 - `bv_clr_bit()`
 - Clears the i th bit in the bit vector. If i is out of range, return false. Otherwise, return true to indicate success.
 - `bv_set_bit()`
 - Sets the i th bit in a bit vector. If i is out of range, return false. Otherwise, return true to indicate success.
 - `bv_print()`
 - A debug function to print the bits of a bit vector. That is, iterate over each of the bits of the bit vector. Print out either 0 or 1 depending on whether each bit is set.

- `bv_length()`
 - Returns the length of a bit vector.
- `bv_create()`
 - The constructor for a bit vector that holds length bits. In the event that sufficient memory cannot be allocated, the function must return NULL. Else, it must return a BitVector *), or a pointer to an allocated BitVector. Each bit of the bit vector should be initialized to 0.
- `parser.h`: Defines the interface for the regex parsing module. Do not modify this.
- `parser.c`: Contains the implementation of the regex parsing module.
- `banhammer.c`: This contains `main()` and may contain the other functions necessary to complete the assignment.
 - Initialize your Bloom filter and hash table.
 - Read in a list of badspeak words with `fscanf()`. Badspeak is simply oldspeak without a newspeak translation. Badspeak is strictly forbidden. Each badspeak word should be added to the Bloom filter and the hash table. The list of proscribed words will be in `badspeak.txt`, which can be found in the resources repository.
 - Read in a list of oldspeak and newspeak pairs with `fscanf()`. Only the oldspeak should be added to the Bloom filter. The oldspeak and newspeak are added to the hash table. The list of oldspeak and

newspeak pairs will be in newspeak.txt, which can also be found in the resources repository.

- Read words in from stdin using the supplied parsing module.
 - For each word that is read in, check to see if it has been added to the Bloom filter. If it has not been added to the Bloom filter, then no action is needed since the word isn't a proscribed word. If the word has most likely been added to the Bloom filter, meaning `bf_probe()` returned true, then further action needs to be taken.
 - If the hash table contains the word and the word does not have a newspeak translation, then the citizen who used this word is guilty of thoughtcrime. Insert this badspeak word into a list of badspeak words that the citizen used in order to notify them of their errors later. What data structure could be used to store these words?
 - If the hash table contains the word, and the word does have a newspeak translation, then the citizen requires counseling on proper Rightspeak. Insert this oldspeak word into a list of oldspeak words with newspeak translations in order to notify the citizen of the revisions needed to be made in order to practice Rightspeak.
 - If the hash table does not contain the word, then all is good since the Bloom filter issued a false positive. No disciplinary action needs to be taken.
- Makefile
 - CC = clang must be specified.

- • CFLAGS = -Wall -Wextra -Werror -Wpedantic must be specified.
- • pkg-config to locate compilation and include flags for the GMP library must be used.
- • make must build the encrypt, decrypt, and keygen executables, as should make all.
- • make decrypt should build only the decrypt program.
- • make encrypt should build only the encrypt program.
- • make keygen should build only the keygen program.
- • make clean must remove all files that are compiler generated.
- • make format should format all your source code, including the header files.
- README.md
 - In proper Markdown syntax.
 - Describe how to use your program and Makefile
 - Lists and explains any command-line options that program accepts.
 - Reports any false positives by scan-build
 - Notes down any known bugs or errors in this file as well for the graders.
- WRITEUP.pdf: This document must be a PDF. The writeup must include at least the following:
 - Graphs comparing the total number of lookups and average binary search tree branches traversed as you vary the hash table and Bloom filter size.

- How do the heights of the binary search trees change?
- What are some factors that can change the height of a binary search tree?
- How does changing the Bloom filter size affect the number of lookups performed in the hash table?
- Analysis of the graphs you produce.
- DESIGN.pdf
 - This document must be a proper PDF.
 - Describes design and design process for program with enough detail such that a sufficiently knowledgeable programmer would be able to replicate implementation.